PropOS uni-app 专家级开发指南

元信息	值
版本	v1.0
日期	2026-04-15
适用对象	前端开发者（中高级）
技术栈	uni-app 4.x · Vue 3 · TypeScript · Pinia · wot-design-uni · luch-request
前置知识	Vue 3 Composition API · TypeScript · Pinia · CSS 变量
项目目录	`app/`

目录

一、架构总览

- 1.1 系统定位
- 1.2 架构层次图
- 1.3 核心原则

二、环境搭建与工程配置

- 2.1 前置工具链
- 2.2 本地开发启动
- 2.3 关键配置文件
- 2.4 Vite 代理配置

三、目录结构详解

- 3.1 命名规范

四、核心分层架构与数据流

- 4.1 严格单向数据流
- 4.2 创建完整功能模块的文件依赖顺序

五、API 层开发规范

- 5.1 HTTP 客户端（client.ts）
- 5.2 API 模块编写范式
- 5.3 API 模块桶导出
- 5.4 禁止事项

六、Store 层开发规范

- 6.1 固定结构模板

6.2 State 固定字段
6.3 错误处理标准模式
6.4 禁止事项
七、Page / Component 层开发规范
7.1 Page 唯一脚本形式
7.2 Page 标准结构
7.3 Page 与 Component 职责边界
7.4 文件复杂度控制
7.5 页面参数接收
7.6 生命周期钩子选择
八、TypeScript 类型体系
8.1 信封类型（已内置）
8.2 业务类型定义规范
8.3 类型使用规则
九、路由与导航
9.1 页面注册
9.2 导航 API
9.3 路由守卫
9.4 页面间传参
十、跨平台适配与条件编译
10.1 条件编译语法
10.2 平台标识符速查
10.3 禁止的平台判断方式
十一、UI 规范与设计体系
11.1 CSS 变量（全局 Token）
11.2 状态色语义映射
11.3 状态标签组件示例
11.4 颜色使用禁令
11.5 布局单位
十二、Composables 编写指南
12.1 什么时候用 Composable
12.2 通用列表 Composable 示例
12.3 在页面中使用
十三、常量管理
13.1 常量文件分类
13.2 使用方式

13.3 新增常量流程
十四、错误处理体系
14.1 三层错误处理链
14.2 各层职责
14.3 错误码业务判断
十五、安全规范
15.1 Token 存储
15.2 敏感数据脱敏
15.3 XSS 防护
15.4 请求安全
十六、性能优化
16.1 列表性能
16.2 网络优化
16.3 包体积优化
16.4 日期处理性能
十七、测试策略
17.1 类型检查
17.2 测试层级
17.3 可测试性设计
十八、新功能模块开发 SOP
Step 1：定义类型
Step 2：添加 API 路径常量
Step 3：编写 API 模块
Step 4：桶导出
Step 5：创建 Store
Step 6：编写页面
Step 7：注册路由
Step 8：类型检查
十九、常见问题与排错
Q1：页面白屏，控制台无报错
Q2：API 请求 404
Q3：类型错误 `Property 'xxx' does not exist on type 'unknown'`
Q4：小程序中 `uni.getStorageSync` 返回空字符串
Q5：H5 跨域报错
Q6：`dayjs` 显示的时间与预期不一致
Q7：wot-design-uni 组件样式不生效

Q8：Store 数据在页面切换后丢失
二十、附录
A. 依赖清单
B. 命令速查
C. 核心文件快速索引
D. 参考文档

一、架构总览

1.1 系统定位

PropOS（Property Operating System）移动端基于 **uni-app 4.x** 构建，一套代码同时运行在：

平台	运行时	说明
iOS	App	运营人员移动办公主力端
Android	App	运营人员移动办公主力端
HarmonyOS Next	App	鸿蒙原生兼容
微信小程序	MP-Weixin	精简版，租户/工单场景
H5	浏览器	开发调试 + 外部门户

1.2 架构层次图

```
┌──────────────────────────────────────────────────────────────┐
│                        Page / Component                      │
│          <template>  ←  只读取 store.state / computed        │
│          <script setup lang="ts">  ←  调用 store.action      │
├──────────────────────────────────────────────────────────────┤
│                         Composables                          │
│          可复用逻辑，跨页面提取（useXxx）                     │
├──────────────────────────────────────────────────────────────┤
│                      Store（Pinia setup）                    │
│          state = ref · getters = computed · actions = async fn │
│          唯一的 HTTP 调用入口层                               │
├──────────────────────────────────────────────────────────────┤
│                        API Modules                           │
│          按业务域封装 apiGet/apiPost/apiPatch/apiDelete       │
├──────────────────────────────────────────────────────────────┤
│                     API Client (luch-request)                │
│          JWT 注入 · 信封解析 · Token 刷新 · 错误统一转换      │
├──────────────────────────────────────────────────────────────┤
│                       Backend REST API                       │
└──────────────────────────────────────────────────────────────┘
```

1.3 核心原则

#	原则	说明
1	单向数据流	`API → Store → Page/Component`，禁止反向
2	**Store 是唯一的 HTTP 调用层**	Page/Component 禁止 import API 函数或直接 fetch
3	常量集中管理	任何路径、阈值、魔法数字必须放 `constants/`
4	**TypeScript 严格模式**	`strict: true`，禁用 `any`
5	**CSS 变量驱动 UI**	语义色通过 CSS 自定义属性传递，不内联 style
6	条件编译做平台差异	禁止 `navigator.userAgent` 判断平台

二、环境搭建与工程配置

2.1 前置工具链

```
# Node.js 18+
node --version  # v18.x 或 v20.x

# 包管理器（推荐 npm，uni-app CLI 项目默认使用 npm）
npm --version

# TypeScript（项目 devDep 已包含，无需全局安装）
```

2.2 本地开发启动

```
# 进入 uni-app 项目目录
cd app

# 安装依赖
npm install

# H5 开发模式（推荐日常开发调试）
npm run dev:h5

# 微信小程序开发模式（产出目录：dist/dev/mp-weixin）
npm run dev:mp-weixin

# 生产构建
npm run build:h5
npm run build:mp-weixin
```

2.3 关键配置文件

文件	作用	注意事项
`vite.config.ts`	Vite 构建 + 路径别名 + API 代理	`@` 指向 `src/`，代理 `/api` 到后端
`tsconfig.json`	TypeScript 编译	`strict: true`，`paths` 配置 `@/*`
`pages.json`	页面路由 + TabBar + 全局样式	**所有页面必须在此注册**
`manifest.json`	平台配置（App/MP/H5）	appid、权限、SDK 配置
`uni.scss`	uni-app 内置样式变量	全局可用，无需 import

2.4 Vite 代理配置

```
// vite.config.ts
export default defineConfig({
  plugins: [uni()],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src'),
    },
  },
  server: {
    proxy: {
      '/api': {
        target: process.env.VITE_API_BASE_URL ?? 'http://localhost:8080',
        changeOrigin: true,
      },
    },
  },
})
```

> 重要：生产环境不走 Vite 代理，移动端通过 `manifest.json` 或环境变量 `VITE_API_BASE_URL` 配置后端地址。

三、目录结构详解

```
app/src/
├── api/                          # HTTP API 层
│   ├── client.ts                 # luch-request 封装，核心 HTTP 客户端
│   ├── index.ts                  # 桶导出（统一出口）
│   └── modules/                  # 按业务域拆分的 API 函数
│       ├── auth.ts               # 认证：登录/登出/刷新/用户信息
│       ├── assets.ts             # M1 资产：楼栋/楼层/房源
│       ├── contracts.ts          # M2 合同：合同/租客/递增模板
│       ├── finance.ts            # M3 财务：账单/收款/NOI/KPI
```

```
│       ├── workorders.ts        # M4 工单：工单/供应商
│       └── subleases.ts         # M5 二房东：穿透管理
│
├── constants/                   # 常量定义（禁止业务代码硬编码）
│   ├── api_paths.ts             # 所有 API 端点路径
│   ├── business_rules.ts        # 业务规则阈值（预警天数、逾期节点等）
│   └── ui_constants.ts          # UI 常量（分页大小、动画时长等）
│
├── stores/                      # Pinia Store（setup 风格）
│   ├── index.ts                 # 桶导出
│   ├── auth.ts                  # 认证状态
│   ├── assets.ts                # 资产模块状态
│   ├── contracts.ts             # 合同模块状态
│   ├── finance.ts               # 财务模块状态
│   ├── workorders.ts            # 工单模块状态
│   └── subleases.ts             # 二房东模块状态
│
├── types/                       # TypeScript 接口定义
│   └── api.ts                   # 信封类型、ApiError、错误码枚举
│
├── composables/                 # 可复用 Composition API 函数
│   ├── useList.ts               # 通用列表（分页、搜索、刷新）
│   ├── useForm.ts               # 通用表单（校验、提交、重置）
│   └── usePermission.ts         # 权限判断
│
├── components/                  # 全局共享组件
│   ├── StatusTag.vue            # 状态标签（语义色映射）
│   ├── EmptyState.vue           # 空状态占位
│   ├── ErrorBlock.vue           # 错误提示块
│   └── PullRefreshList.vue      # 下拉刷新 + 触底加载列表
│
├── pages/                       # 页面（按 pages.json 注册）
│   ├── auth/login.vue           # 登录页
│   ├── dashboard/index.vue      # 总览仪表盘
│   ├── assets/                  # M1 资产模块页面
│   ├── contracts/               # M2 合同模块页面
│   ├── finance/                 # M3 财务模块页面
│   ├── workorders/              # M4 工单模块页面
│   └── subleases/               # M5 二房东模块页面
│
├── static/                      # 静态资源（图片、图标）
│   └── tabbar/                  # TabBar 图标（常态 + 选中态）
│
├── App.vue                      # 根组件（路由守卫 + 全局 CSS 变量）
├── main.ts                      # 入口（createSSRApp + Pinia）
├── pages.json                   # 路由配置 + TabBar + 全局样式
├── manifest.json                # 平台配置
```

```
├── uni.scss                    # uni-app 内置样式变量覆盖
└── env.d.ts                    # Vite + Vue 类型声明
```

3.1 命名规范

类别	规范	示例
文件夹	`kebab-case`	`api-paths`，`floor-plan`
Vue 组件文件	`kebab-case.vue` 或 `PascalCase.vue`	`login.vue`，`StatusTag.vue`
TS 文件	`camelCase.ts` 或域名 `kebab-case.ts`	`client.ts`，`api_paths.ts`
变量/函数	`camelCase`	`fetchList`，`isLoggedIn`
类型/接口	`PascalCase`	`ApiResponse`，`PaginationMeta`
常量	`SCREAMING_SNAKE_CASE`	`API_AUTH_LOGIN`，`DEFAULT_PAGE_SIZE`
CSS 变量	`--kebab-case`	`--color-primary`，`--color-danger`

四、核心分层架构与数据流

4.1 严格单向数据流

```
用户交互 → Page 调用 store.action() → Store 调用 api 函数 → API Client 发 HTTP
                                                                    ↓
用户看到 ← Page 读取 store.state  ← Store 更新 state     ← API Client 返回 data
```

铁律：

层级	可以做	禁止做
Page/Component	读 store.state、调 store.action	直接调 api 函数、内联 HTTP 请求
Store	调 api 函数、管理 state	直接操作 DOM、访问 uni API（除导航）
API Module	调 client 辅助函数	直接 new Request()、硬编码路径
API Client	管理 token、解析信封、转换错误	包含业务逻辑

4.2 创建完整功能模块的文件依赖顺序

```
1. types/xxx.ts           ← 定义接口（无依赖）
2. constants/api_paths.ts ← 添加路径常量（无依赖）
3. api/modules/xxx.ts     ← 实现 API 函数（依赖 1 + 2 + client）
4. api/index.ts           ← 桶导出（依赖 3）
```

```
5. stores/xxx.ts          ← 实现 Store（依赖 3 + 1）
6. stores/index.ts        ← 桶导出（依赖 5）
7. components/XxxCard.vue ← 子组件（依赖 1，不依赖 store）
8. pages/xxx/index.vue    ← 页面（依赖 5 + 7）
9. pages.json             ← 注册路由
```

五、API 层开发规范

5.1 HTTP 客户端（client.ts）

客户端基于 `luch-request` 封装，已实现以下能力：

能力	实现
JWT 自动注入	请求拦截器从 `uni.getStorageSync('access_token')` 读取
信封自动解析	`apiGet<T>` 返回 `T`，不是 `ApiResponse<T>`
401 自动刷新	响应拦截器检测 401 → 调 `refresh` → 重发
错误统一转换	所有错误抛出 `ApiError(code, message, statusCode)`
超时配置	默认 15000ms

已封装的辅助函数：

```
// 获取单对象（自动解封 data 字段）
apiGet<T>(url: string, params?: Record<string, unknown>): Promise<T>

// 获取分页列表（返回 data + meta）
apiGetList<T>(url: string, params?: Record<string, unknown>): Promise<ApiListResponse<T>>

// 创建资源
apiPost<T>(url: string, data?: HttpData): Promise<T>

// 局部更新
apiPatch<T>(url: string, data?: HttpData): Promise<T>

// 删除资源
apiDelete(url: string): Promise<void>
```

5.2 API 模块编写范式

每个业务模块对应一个 `api/modules/<domain>.ts` 文件：

```
// api/modules/contracts.ts

import { apiGet, apiGetList, apiPost, apiPatch, apiDelete } from '@/api/client'
import {
```

```
  API_CONTRACTS,
  API_TENANTS,
} from '@/constants/api_paths'
import type { Contract, ContractDetail, Tenant } from '@/types/contract'
import type { ApiListResponse } from '@/types/api'

export interface ContractListParams {
  page?: number
  pageSize?: number
  status?: string
  buildingId?: string
  keyword?: string
}

export const contractsApi = {
  /** 合同列表（分页） */
  list: (params?: ContractListParams) =>
    apiGetList<Contract>(API_CONTRACTS, params as Record<string, unknown>),

  /** 合同详情 */
  detail: (id: string) =>
    apiGet<ContractDetail>(`${API_CONTRACTS}/${id}`),

  /** 新建合同 */
  create: (data: Partial<Contract>) =>
    apiPost<Contract>(API_CONTRACTS, data),

  /** 更新合同 */
  update: (id: string, data: Partial<Contract>) =>
    apiPatch<Contract>(`${API_CONTRACTS}/${id}`, data),

  /** 删除合同 */
  remove: (id: string) =>
    apiDelete(`${API_CONTRACTS}/${id}`),

  /** 合同状态变更（审批/终止/续约） */
  changeStatus: (id: string, action: string, reason?: string) =>
    apiPost<Contract>(`${API_CONTRACTS}/${id}/actions/${action}`, { reason }),
}
```

5.3 API 模块桶导出

```
// api/index.ts
export { authApi } from './modules/auth'
export { assetsApi } from './modules/assets'
export { contractsApi } from './modules/contracts'
export { financeApi } from './modules/finance'
export { workordersApi } from './modules/workorders'
export { subleasesApi } from './modules/subleases'
```

5.4 禁止事项

#	禁止	正确做法
1	在 store/component 中直接 `uni.request()`	通过 `apiGet/apiPost`
2	硬编码路径 `'/api/contracts'`	从 `@/constants/api_paths` 导入
3	直接 `new Request()`	使用 `client.ts` 已有实例
4	在 API 函数中写业务逻辑	API 函数仅做 HTTP 调用 + 参数透传
5	透传 `luch-request` 原始错误对象	已由 client 统一转为 `ApiError`

六、Store 层开发规范

6.1 固定结构模板

所有业务 Store 必须使用 Pinia setup 风格，并遵循以下标准结构：

```
// stores/contracts.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { contractsApi } from '@/api/modules/contracts'
import type { Contract, ContractDetail } from '@/types/contract'
import type { PaginationMeta } from '@/types/api'
import { ApiError } from '@/types/api'
import { DEFAULT_PAGE_SIZE } from '@/constants/ui_constants'

export const useContractsStore = defineStore('contracts', () => {
  // ═══════════════ State ═══════════════
  const list = ref<Contract[]>([])
  const item = ref<ContractDetail | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)
  const meta = ref<PaginationMeta | null>(null)

  // ═══════════════ Getters ═══════════════
  const total = computed(() => meta.value?.total ?? 0)
  const isEmpty = computed(() => list.value.length === 0 && !loading.value)

  // ═══════════════ Actions ═══════════════

  /** 获取合同列表 */
  async function fetchList(params?: { page?: number; status?: string }) {
    loading.value = true
    error.value = null
    try {
      const res = await contractsApi.list({
        page: params?.page ?? 1,
```

```
        pageSize: DEFAULT_PAGE_SIZE,
        status: params?.status,
      })
      list.value = res.data
      meta.value = res.meta
    } catch (e) {
      error.value = e instanceof ApiError ? e.message : '获取合同列表失败'
    } finally {
      loading.value = false
    }
  }

  /** 获取合同详情 */
  async function fetchDetail(id: string) {
    loading.value = true
    error.value = null
    try {
      item.value = await contractsApi.detail(id)
    } catch (e) {
      error.value = e instanceof ApiError ? e.message : '获取合同详情失败'
    } finally {
      loading.value = false
    }
  }

  /** 重置状态 */
  function $reset() {
    list.value = []
    item.value = null
    loading.value = false
    error.value = null
    meta.value = null
  }

  return {
    // State
    list, item, loading, error, meta,
    // Getters
    total, isEmpty,
    // Actions
    fetchList, fetchDetail, $reset,
  }
})
```

6.2 State 固定字段

字段	类型	用途
`list`	`ref<T[]>`	列表数据
`item`	`ref<T \ null>`	
`loading`	`ref<boolean>`	加载态（控制骨架屏 / 按钮禁用）
`error`	`ref<string \ null>`	
`meta`	`ref<PaginationMeta \ null>`	

> 如果模块复杂度过高（action > 8 个或 state > 10 个），按子领域拆分，如 `useContractListStore` + `useContractFormStore` 。

6.3 错误处理标准模式

```
// ✅ 正确：统一错误处理
try {
  const res = await contractsApi.list(params)
  list.value = res.data
} catch (e) {
  error.value = e instanceof ApiError ? e.message : '操作失败，请重试'
}

// ❌ 错误：透传原始错误
catch (e) {
  error.value = e  // 不允许！可能泄漏内部信息
}

// ❌ 错误：不处理错误
// 不写 try/catch 直接调用 API
```

6.4 禁止事项

#	禁止	正确做法
1	`defineStore('id', { state, actions })` options 风格	setup 风格 `defineStore('id', () => {})`
2	在 store 中操作 DOM	仅管理数据状态
3	从 store 透传原始 Error 对象	`error.value` 只存 string
4	在 store 中引入 UI 框架组件	Toast/Dialog 等 UI 反馈在 Page 层做

七、Page / Component 层开发规范

7.1 Page 唯一脚本形式

```
<script setup lang="ts">
// ✅ 唯一允许的脚本形式
import { onMounted, computed } from 'vue'
import { useContractsStore } from '@/stores/contracts'

const store = useContractsStore()

onMounted(() => {
  store.fetchList()
})
</script>
```

```
<!-- ❌ 禁止使用 Options API -->
<script lang="ts">
export default {
  data() { return {} },
  methods: {},
}
</script>
```

7.2 Page 标准结构

```
<template>
  <!-- 加载态 -->
  <view v-if="store.loading" class="loading-container">
    <wd-loading />
  </view>

  <!-- 错误态 -->
  <view v-else-if="store.error" class="error-container">
    <ErrorBlock :message="store.error" @retry="store.fetchList()" />
  </view>

  <!-- 空态 -->
  <view v-else-if="store.isEmpty" class="empty-container">
    <EmptyState text="暂无合同" />
  </view>

  <!-- 正常内容 -->
  <view v-else class="page-content">
    <view
      v-for="contract in store.list"
      :key="contract.id"
      class="contract-card"
      @tap="goDetail(contract.id)"
    >
```

```
      <text class="contract-no">{{ contract.contractNo }}</text>
      <StatusTag :status="contract.status" />
    </view>
  </view>
</template>

<script setup lang="ts">
import { onMounted } from 'vue'
import { useContractsStore } from '@/stores/contracts'
import StatusTag from '@/components/StatusTag.vue'
import EmptyState from '@/components/EmptyState.vue'
import ErrorBlock from '@/components/ErrorBlock.vue'

const store = useContractsStore()

onMounted(() => {
  store.fetchList()
})

function goDetail(id: string) {
  uni.navigateTo({ url: `/pages/contracts/detail?id=${id}` })
}
</script>
```

7.3 Page 与 Component 职责边界

场景	Page 负责	Component 负责
数据获取	调 `store.fetchList()`	不调 store action
导航跳转	`uni.navigateTo(...)`	emit 事件给 Page
Toast/Dialog	`uni.showToast(...)`	emit 结果给 Page
业务判断	使用 `computed` 或 composable	通过 props 接收渲染即可

7.4 文件复杂度控制

信号	拆分策略
`.vue` 文件 > 250 行	子区域提取到 `components/` 下独立组件
`<template>` 嵌套 > 4 层	提取嵌套部分为子组件
页面同时含列表 + 表单 + 图表	每个区域一个组件，页面只做组合
`v-for` 内部逻辑过重	列表项提取为 `XxxCard.vue`

7.5 页面参数接收

```
<script setup lang="ts">
import { onLoad } from '@dcloudio/uni-app'
import { ref } from 'vue'
import { useContractsStore } from '@/stores/contracts'

const store = useContractsStore()
const contractId = ref('')

onLoad((query) => {
  contractId.value = query?.id ?? ''
  if (contractId.value) {
    store.fetchDetail(contractId.value)
  }
})
</script>
```

7.6 生命周期钩子选择

钩子	使用场景	来源
`onLoad(query)`	页面首次加载，获取路由参数	`@dcloudio/uni-app`
`onShow()`	页面每次显示（含返回），刷新数据	`@dcloudio/uni-app`
`onReady()`	DOM 已渲染，操作节点	`@dcloudio/uni-app`
`onPullDownRefresh()`	下拉刷新	`@dcloudio/uni-app`
`onReachBottom()`	触底加载更多	`@dcloudio/uni-app`
`onMounted()`	组件挂载（非页面级场景）	`vue`

> 区别：Page 优先使用 uni-app 生命周期（ `onLoad` / `onShow` ），Component 使用 Vue 生命周期（ `onMounted` / `onUnmounted` ）。

八、TypeScript 类型体系

8.1 信封类型（已内置）

```
// types/api.ts — 已实现，直接引用

/** 单对象响应 */
interface ApiResponse<T> {
  data: T
  meta?: PaginationMeta
}
```

```
/** 分页列表响应 */
interface ApiListResponse<T> {
  data: T[]
  meta: PaginationMeta
}

/** 分页元信息 */
interface PaginationMeta {
  page: number
  pageSize: number
  total: number
}

/** 业务异常 */
class ApiError extends Error {
  constructor(
    public readonly code: string,
    message: string,
    public readonly statusCode?: number,
  ) { super(message); this.name = 'ApiError' }
}
```

8.2 业务类型定义规范

每个模块在 `types/` 下创建独立文件：

```
// types/contract.ts

/** 合同列表项（列表接口返回） */
export interface Contract {
  id: string
  contractNo: string
  tenantName: string
  unitNo: string
  status: ContractStatus
  startDate: string     // ISO 8601
  endDate: string       // ISO 8601
  monthlyRent: number   // 单位：元（后端返回整数分时需前端 /100）
}

/** 合同详情（详情接口返回，字段更多） */
export interface ContractDetail extends Contract {
  tenant: TenantBrief
  unit: UnitBrief
  escalationRules: EscalationRule[]
  deposits: Deposit[]
  invoices: InvoiceBrief[]
  createdAt: string
  updatedAt: string
}

/** 合同状态枚举 */
```

```
export type ContractStatus =
  | 'draft'
  | 'pending_approval'
  | 'active'
  | 'expiring_soon'
  | 'expired'
  | 'terminated'
  | 'renewed'
```

8.3 类型使用规则

规则	说明
禁用 `any`	使用 `unknown` + 类型收窄
接口定义集中	放 `src/types/`，不在组件内定义
使用信封类型	API 返回必须声明 `ApiResponse<T>` 或 `ApiListResponse<T>`
字符串联合代替 enum	`type Status = 'active' \
日期字段为 string	ISO 8601 格式，展示时 `dayjs()` 转换

九、路由与导航

9.1 页面注册

所有页面必须在 `pages.json` 中注册，否则运行时找不到页面。

```
{
  "pages": [
    {
      "path": "pages/contracts/index",
      "style": { "navigationBarTitleText": "合同管理" }
    },
    {
      "path": "pages/contracts/detail",
      "style": { "navigationBarTitleText": "合同详情" }
    }
  ]
}
```

9.2 导航 API

API	用途	场景
`uni.navigateTo({ url })`	保留当前页，跳新页	列表 → 详情
`uni.redirectTo({ url })`	关闭当前页，跳新页	表单提交后回列表
`uni.navigateBack()`	返回上一页	详情 → 列表
`uni.switchTab({ url })`	切换 TabBar 页	底部导航切换
`uni.reLaunch({ url })`	关闭所有页面，开新页	退出登录 → 登录页

```
// ✅ 正确：使用 uni 导航 API
uni.navigateTo({ url: `/pages/contracts/detail?id=${id}` })

// ❌ 禁止：使用 HTML 原生方式
// <a href="/pages/contracts/detail">
// history.push(...)
// router.push(...)  ← uni-app 没有 vue-router
```

9.3 路由守卫

在 `App.vue` 中通过 `uni.addInterceptor` 实现全局路由守卫：

```
const PUBLIC_PAGES = ['/pages/auth/login']

;(['navigateTo', 'redirectTo', 'reLaunch', 'switchTab'] as const).forEach((method) => {
  uni.addInterceptor(method, {
    invoke(args: { url: string }) {
      if (PUBLIC_PAGES.some(p => args.url.startsWith(p))) return true
      const token = uni.getStorageSync('access_token')
      if (!token) {
        uni.reLaunch({ url: '/pages/auth/login' })
        return false
      }
      return true
    },
  })
})
```

9.4 页面间传参

```
// 发送方
uni.navigateTo({
  url: `/pages/contracts/detail?id=${contract.id}&from=list`
})

// 接收方
```

```
onLoad((query) => {
  const id = query?.id ?? ''
  const from = query?.from ?? ''
})
```

> 注意：参数通过 URL query string 传递，仅支持字符串。复杂对象应通过 store 共享或 `uni.$emit` / `uni.$on` 事件传递。

十、跨平台适配与条件编译

10.1 条件编译语法

uni-app 使用独有的条件编译指令，**编译期**剔除不相关平台代码：

```
<!-- 模板中 -->
<!-- #ifdef MP-WEIXIN -->
<button open-type="getPhoneNumber" @getphonenumber="onGetPhone">
 微信授权手机号
</button>
<!-- #endif -->

<!-- #ifdef APP-PLUS -->
<view @tap="scanQR">扫一扫</view>
<!-- #endif -->

<!-- #ifndef H5 -->
<!-- 除 H5 外所有平台显示 -->
<NativeComponent />
<!-- #endif -->
```

```
// 脚本中
// #ifdef MP-WEIXIN
wx.login({ success: (res) => { /* 微信登录 */ } })
// #endif

// #ifdef H5
console.log('H5 环境')
// #endif
```

```
/* 样式中 */
/* #ifdef MP-WEIXIN */
.container { padding-bottom: env(safe-area-inset-bottom); }
/* #endif */
```

10.2 平台标识符速查

标识符	平台
`APP-PLUS`	iOS / Android App
`APP-HARMONY`	HarmonyOS Next
`MP-WEIXIN`	微信小程序
`H5`	H5 浏览器
`APP-PLUS \	\

10.3 禁止的平台判断方式

```
// ❌ 禁止：运行时判断
if (navigator.userAgent.includes('MicroMessenger')) { ... }
if (process.env.UNI_PLATFORM === 'mp-weixin') { ... }

// ✅ 正确：编译期条件编译
// #ifdef MP-WEIXIN
// 微信专属逻辑
// #endif
```

十一、UI 规范与设计体系

11.1 CSS 变量（全局 Token）

在 `App.vue` `<style>` 中定义，所有页面自动继承：

```
page {
  --color-success: #52c41a;   /* leased / paid — 已租 / 已核销 */
  --color-warning: #faad14;   /* expiring_soon — 即将到期 / 预警 */
  --color-danger: #ff4d4f;    /* vacant / overdue — 空置 / 逾期 */
  --color-neutral: #8c8c8c;   /* non_leasable — 非可租区域 */
  --color-primary: #1677ff;   /* 主品牌色 */
}
```

11.2 状态色语义映射

业务状态	语义色	CSS 变量	wot-design type
`leased` / `paid` / `active`	成功（绿）	`var(--color-success)`	`success`
`expiring_soon` / `warning`	预警（黄/橙）	`var(--color-warning)`	`warning`
`vacant` / `overdue` / `error` / `terminated`	危险（红）	`var(--color-danger)`	`error`
`non_leasable` / `draft`	中性（灰）	`var(--color-neutral)`	`info`

11.3 状态标签组件示例

```
<!-- components/StatusTag.vue -->
<template>
  <wd-tag :type="tagType" size="small">{{ label }}</wd-tag>
</template>

<script setup lang="ts">
import { computed } from 'vue'

const props = defineProps<{
  status: string
  label?: string
}>()

const STATUS_MAP: Record<string, { type: string; label: string }> = {
  active:         { type: 'success', label: '生效中' },
  leased:         { type: 'success', label: '已租' },
  paid:           { type: 'success', label: '已付' },
  expiring_soon:  { type: 'warning', label: '即将到期' },
  pending:        { type: 'warning', label: '待处理' },
  vacant:         { type: 'error',   label: '空置' },
  overdue:        { type: 'error',   label: '逾期' },
  terminated:     { type: 'error',   label: '已终止' },
  draft:          { type: 'info',    label: '草稿' },
  non_leasable:   { type: 'info',    label: '非可租' },
}

const entry = computed(() => STATUS_MAP[props.status] ?? { type: 'info', label: props.status })
const tagType = computed(() => entry.value.type)
const label = computed(() => props.label ?? entry.value.label)
</script>
```

11.4 颜色使用禁令

```
<!-- ❌ 禁止：内联 style 写颜色 -->
<text style="color: green">已租</text>
```

```
<text style="color: red">逾期</text>
<view style="background: #52c41a"></view>

<!-- ✅ 正确：使用 CSS 变量 -->
<text class="status-success">已租</text>
<text class="status-danger">逾期</text>

<!-- ✅ 正确：使用 wot-design-uni 组件 type prop -->
<wd-tag type="success">已租</wd-tag>
<wd-tag type="error">逾期</wd-tag>
```

```
.status-success { color: var(--color-success); }
.status-danger { color: var(--color-danger); }
```

11.5 布局单位

单位	使用场景
`rpx`	尺寸、间距、字号（自动适配屏幕宽度）
`px`	1px 边框、图片固定尺寸
`%` / `vh` / `vw`	全屏布局、百分比布局

```
/* ✅ 推荐 */
.card {
  padding: 24rpx 32rpx;
  font-size: 28rpx;
  border: 1px solid #e8e8e8;
  border-radius: 16rpx;
}
```

十二、Composables 编写指南

12.1 什么时候用 Composable

场景	用 Composable	用 Store
跨页面共享的服务端数据		✅
可复用的 UI 交互逻辑（下拉刷新、搜索节流）	✅	
格式化/转换函数	✅	
权限判断	✅	

12.2 通用列表 Composable 示例

```
// composables/useList.ts
import { ref, computed } from 'vue'
import type { PaginationMeta } from '@/types/api'
import { DEFAULT_PAGE_SIZE } from '@/constants/ui_constants'

export function useList<T>(
  fetchFn: (params: { page: number; pageSize: number }) => Promise<{ data: T[]; meta:
PaginationMeta }>,
) {
  const list = ref<T[]>([]) as Ref<T[]>
  const loading = ref(false)
  const error = ref<string | null>(null)
  const meta = ref<PaginationMeta | null>(null)
  const page = ref(1)

  const hasMore = computed(() => {
    if (!meta.value) return true
    return page.value * DEFAULT_PAGE_SIZE < meta.value.total
  })

  async function refresh() {
    page.value = 1
    loading.value = true
    error.value = null
    try {
      const res = await fetchFn({ page: 1, pageSize: DEFAULT_PAGE_SIZE })
      list.value = res.data
      meta.value = res.meta
    } catch (e) {
      error.value = e instanceof Error ? e.message : '加载失败'
    } finally {
      loading.value = false
    }
  }
```

```
  async function loadMore() {
    if (!hasMore.value || loading.value) return
    page.value++
    loading.value = true
    try {
      const res = await fetchFn({ page: page.value, pageSize: DEFAULT_PAGE_SIZE })
      list.value.push(...res.data)
      meta.value = res.meta
    } catch (e) {
      page.value--
      error.value = e instanceof Error ? e.message : '加载失败'
    } finally {
      loading.value = false
    }
  }

  return { list, loading, error, meta, hasMore, refresh, loadMore }
}
```

12.3 在页面中使用

```
<script setup lang="ts">
import { onShow } from '@dcloudio/uni-app'
import { useList } from '@/composables/useList'
import { contractsApi } from '@/api/modules/contracts'

const { list, loading, hasMore, refresh, loadMore } = useList(contractsApi.list)

onShow(() => refresh())
onPullDownRefresh(async () => { await refresh(); uni.stopPullDownRefresh() })
onReachBottom(() => loadMore())
</script>
```

十三、常量管理

13.1 常量文件分类

文件	内容	示例
`constants/api_paths.ts`	所有 API 端点路径	`API_CONTRACTS = '/api/contracts'`
`constants/business_rules.ts`	业务规则阈值	`LEASE_EXPIRY_WARNING_DAYS_90 = 90`
`constants/ui_constants.ts`	UI 展示参数	`DEFAULT_PAGE_SIZE = 20`

13.2 使用方式

```
// ✅ 正确
```

```
import { API_CONTRACTS } from '@/constants/api_paths'
import { DEFAULT_PAGE_SIZE } from '@/constants/ui_constants'

// ❌ 禁止：硬编码
const url = '/api/contracts'        // 魔法字符串
const pageSize = 20                 // 魔法数字
```

13.3 新增常量流程

1. 确定常量类别（API 路径 / 业务规则 / UI 参数）
2. 在对应文件中添加，使用 `SCREAMING_SNAKE_CASE` 命名
3. 添加 JSDoc 注释说明含义
4. 全局搜索确认没有已存在的等价常量

十四、错误处理体系

14.1 三层错误处理链

```
API Client (luch-request 拦截器)
  ↓  统一转为 ApiError(code, message, statusCode)
Store (try/catch)
  ↓  error.value = e instanceof ApiError ? e.message : '...'
Page (模板读取)
  ↓  <ErrorBlock :message="store.error" @retry="..." />
```

14.2 各层职责

层	输入	输出	职责
API Client	HTTP 响应	`ApiError`	解析信封、转换错误类型、401 刷新
Store	`ApiError`	`error: ref<string>`	提取 `.message`，不透传对象
Page	`store.error`	UI 渲染	展示错误文案 + 重试按钮

14.3 错误码业务判断

```
import { ApiError, ErrorCode } from '@/types/api'

try {
  await contractsApi.create(data)
} catch (e) {
  if (e instanceof ApiError && e.code === ErrorCode.UNIT_OCCUPIED) {
    error.value = '该房源已被占用，请选择其他房源'
  } else {
    error.value = e instanceof ApiError ? e.message : '创建合同失败'
```

```
  }
}
```

十五、安全规范

15.1 Token 存储

```
// ✅ 使用 uni 存储（各平台适配）
uni.setStorageSync('access_token', token)
uni.getStorageSync('access_token')
uni.removeStorageSync('access_token')

// ❌ 禁止：localStorage / cookie（小程序不可用）
localStorage.setItem('token', token)
```

15.2 敏感数据脱敏

后端返回的证件号、手机号已脱敏（仅后 4 位），前端不做额外脱敏处理，直接展示即可。

```
// 后端返回：{ idCard: '****1234', phone: '****5678' }
// 前端直接显示，不尝试还原或自行脱敏
```

15.3 XSS 防护

- `v-html` 禁止用于渲染用户输入内容
- 使用 `{{ }}` 插值（Vue 自动转义）
- 富文本场景使用 `<rich-text>` 组件（uni-app 内置，已做安全处理）

15.4 请求安全

- 所有 HTTP 通过 API Client 统一发出，JWT 自动注入
- 禁止在 URL query string 中传递敏感参数（ID 除外）
- 文件访问走后端代理 `GET /api/files/{path}`，不暴露存储地址

十六、性能优化

16.1 列表性能

策略	实现
分页加载	默认 `pageSize=20`，触底加载更多
虚拟列表	超过 100 条时考虑使用 `<recycle-list>`（原生端）或三方虚拟滚动
图片懒加载	`<image lazy-load />`
骨架屏	加载态显示 `<wd-skeleton>` 而非空白

16.2 网络优化

策略	实现
防抖搜索	搜索输入 300ms 防抖后再调 API
请求去重	Store 中判断 `loading.value` 为 true 时跳过重复请求
缓存策略	非频繁变动数据（楼栋列表）首次加载后缓存到 store

16.3 包体积优化

策略	说明
按需引入 wot-design-uni	使用 unplugin-auto-import 按需加载
条件编译剔除平台代码	`#ifdef` 确保每个平台只包含自己的代码
静态资源压缩	TabBar 图标使用 PNG8，小 icon 考虑 iconfont

16.4 日期处理性能

```
import dayjs from 'dayjs'

// ✅ 仅展示用：前端格式化
const displayDate = dayjs(contract.startDate).format('YYYY-MM-DD')

// ❌ 禁止：前端做业务日期计算
// const overdueDays = dayjs().diff(dayjs(invoice.dueDate), 'day')
// 逾期天数由后端计算并返回
```

十七、测试策略

17.1 类型检查

```
# 全量类型检查
npm run type-check
# 等同于 vue-tsc --noEmit
```

17.2 测试层级

层级	工具	覆盖范围
类型检查	`vue-tsc`	编译时类型安全
单元测试	Vitest（推荐）	Composables、工具函数
组件测试	@vue/test-utils	关键交互组件
E2E	uni-automator	关键路径（登录 → 列表 → 详情）

17.3 可测试性设计

```
// ✅ 可测试：逻辑在 composable 中，可独立测试
export function useStatusColor(status: Ref<string>) {
  return computed(() => {
    switch (status.value) {
      case 'active': return 'success'
      case 'overdue': return 'error'
      default: return 'info'
    }
  })
}

// ❌ 难测试：逻辑内联在 template 中
// <text :class="status === 'active' ? 'success' : status === 'overdue' ? 'error' : 'info'">
```

十八、新功能模块开发 SOP

以「添加工单模块」为例，严格按照以下步骤顺序执行：

Step 1：定义类型

```
// types/workorder.ts
export interface WorkOrder {
  id: string
  orderNo: string
  title: string
```

```
  status: WorkOrderStatus
  priority: 'low' | 'medium' | 'high' | 'urgent'
  unitId: string
  unitNo: string
  reporterName: string
  assigneeName?: string
  createdAt: string
  // ...
}

export type WorkOrderStatus = 'pending' | 'assigned' | 'in_progress' | 'completed' | 'closed'
```

Step 2：添加 API 路径常量

```
// constants/api_paths.ts
export const API_WORKORDERS = '/api/workorders'
```

Step 3：编写 API 模块

```
// api/modules/workorders.ts
import { apiGet, apiGetList, apiPost, apiPatch } from '@/api/client'
import { API_WORKORDERS } from '@/constants/api_paths'
import type { WorkOrder } from '@/types/workorder'

export const workordersApi = {
  list: (params?: Record<string, unknown>) =>
    apiGetList<WorkOrder>(API_WORKORDERS, params),
  detail: (id: string) =>
    apiGet<WorkOrder>(`${API_WORKORDERS}/${id}`),
  create: (data: Partial<WorkOrder>) =>
    apiPost<WorkOrder>(API_WORKORDERS, data),
  update: (id: string, data: Partial<WorkOrder>) =>
    apiPatch<WorkOrder>(`${API_WORKORDERS}/${id}`, data),
}
```

Step 4：桶导出

```
// api/index.ts — 添加一行
export { workordersApi } from './modules/workorders'
```

Step 5：创建 Store

```
// stores/workorders.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { workordersApi } from '@/api/modules/workorders'
import type { WorkOrder } from '@/types/workorder'
import type { PaginationMeta } from '@/types/api'
import { ApiError } from '@/types/api'
import { DEFAULT_PAGE_SIZE } from '@/constants/ui_constants'
```

```
export const useWorkordersStore = defineStore('workorders', () => {
  const list = ref<WorkOrder[]>([])
  const item = ref<WorkOrder | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)
  const meta = ref<PaginationMeta | null>(null)

  const total = computed(() => meta.value?.total ?? 0)

  async function fetchList(params?: { page?: number; status?: string }) {
    loading.value = true
    error.value = null
    try {
      const res = await workordersApi.list({
        page: params?.page ?? 1,
        pageSize: DEFAULT_PAGE_SIZE,
        status: params?.status,
      })
      list.value = res.data
      meta.value = res.meta
    } catch (e) {
      error.value = e instanceof ApiError ? e.message : '获取工单列表失败'
    } finally {
      loading.value = false
    }
  }

  async function fetchDetail(id: string) {
    loading.value = true
    error.value = null
    try {
      item.value = await workordersApi.detail(id)
    } catch (e) {
      error.value = e instanceof ApiError ? e.message : '获取工单详情失败'
    } finally {
      loading.value = false
    }
  }

  return { list, item, loading, error, meta, total, fetchList, fetchDetail }
})
```

Step 6：编写页面

```
<!-- pages/workorders/index.vue -->
<template>
  <view class="workorder-list">
    <wd-skeleton v-if="store.loading && store.list.length === 0" :count="5" />

    <view v-else-if="store.error">
      <ErrorBlock :message="store.error" @retry="store.fetchList()" />
    </view>
```

```
    <view v-else-if="store.list.length === 0">
      <EmptyState text="暂无工单" />
    </view>

    <view v-else>
      <view
        v-for="order in store.list"
        :key="order.id"
        class="order-card"
        @tap="goDetail(order.id)"
      >
        <view class="order-header">
          <text class="order-no">{{ order.orderNo }}</text>
          <StatusTag :status="order.status" />
        </view>
        <text class="order-title">{{ order.title }}</text>
        <text class="order-unit">{{ order.unitNo }}</text>
      </view>
    </view>
  </view>
</template>

<script setup lang="ts">
import { onShow } from '@dcloudio/uni-app'
import { useWorkordersStore } from '@/stores/workorders'
import StatusTag from '@/components/StatusTag.vue'
import EmptyState from '@/components/EmptyState.vue'
import ErrorBlock from '@/components/ErrorBlock.vue'

const store = useWorkordersStore()

onShow(() => store.fetchList())

function goDetail(id: string) {
  uni.navigateTo({ url: `/pages/workorders/detail?id=${id}` })
}
</script>
```

Step 7：注册路由

```
// pages.json — 添加页面
{
  "path": "pages/workorders/index",
  "style": { "navigationBarTitleText": "工单管理" }
}
```

Step 8：类型检查

```
npm run type-check
```

十九、常见问题与排错

Q1：页面白屏，控制台无报错

原因：页面未在 `pages.json` 中注册。

解决：在 `pages.json` 的 `pages` 数组中添加页面路径。

Q2：API 请求 404

排查：

1. 检查 `constants/api_paths.ts` 中路径是否与后端一致
2. H5 开发模式检查 `vite.config.ts` 代理是否生效
3. 小程序/App 检查 `VITE_API_BASE_URL` 环境变量是否配置

Q3：类型错误 `Property 'xxx' does not exist on type 'unknown'`

原因：`luch-request` 返回类型推断不精确。

解决：确保 API 函数声明了正确的泛型：

```
// ✅ 正确
apiGet<ContractDetail>(`${API_CONTRACTS}/${id}`)

// ❌ 缺少泛型
apiGet(`${API_CONTRACTS}/${id}`)
```

Q4：小程序中 `uni.getStorageSync` 返回空字符串

原因：微信小程序 Storage 限制 10MB，且 key 区分大小写。

解决：确认 key 一致（`access_token`），检查存储是否成功。

Q5：H5 跨域报错

原因：Vite 代理未生效或后端 CORS 未配置。

解决：

1. 确认 `vite.config.ts` 中 `server.proxy` 配置正确
2. 确认后端 `CORS_ORIGINS` 环境变量包含开发地址

Q6：`dayjs` 显示的时间与预期不一致

原因：后端返回 UTC 时间，前端 `dayjs` 默认按本地时区解析。

解决：无需额外处理，`dayjs(isoString).format('YYYY-MM-DD')` 已自动转本地时区。

Q7：wot-design-uni 组件样式不生效

原因：组件未正确引入或 `scoped` 样式无法穿透。

解决：

```
/* 穿透第三方组件样式 */
:deep(.wd-tag) {
  font-size: 24rpx;
}
```

Q8：Store 数据在页面切换后丢失

原因：Pinia store 在内存中，页面 `onShow` 时需重新获取。

解决：

```
// 在 onShow 而非 onLoad 中获取数据，确保每次显示时刷新
onShow(() => {
  store.fetchList()
})
```

二十、附录

A. 依赖清单

包名	版本	用途
`vue`	^3.4.21	视图框架
`pinia`	^2.1.7	状态管理
`wot-design-uni`	^1.6.0	UI 组件库
`luch-request`	^3.1.1	HTTP 客户端（支持全平台）
`dayjs`	^1.11.13	日期格式化
`@dcloudio/uni-app`	3.0.0-408...	uni-app 核心
`typescript`	^5.3.0	类型系统
`vite`	5.2.8	构建工具

B. 命令速查

命令	说明
`npm run dev:h5`	H5 开发模式
`npm run dev:mp-weixin`	微信小程序开发模式
`npm run build:h5`	H5 生产构建
`npm run build:mp-weixin`	微信小程序生产构建
`npm run type-check`	TypeScript 类型检查

C. 核心文件快速索引

需求	文件
修改 API 基础地址	`vite.config.ts` → `server.proxy.target`
新增 API 端点	`constants/api_paths.ts`
新增业务常量	`constants/business_rules.ts`
修改 HTTP 超时	`api/client.ts` → `timeout`
添加全局 CSS 变量	`App.vue` → `<style>`
新增 TabBar	`pages.json` → `tabBar.list`
修改全局导航栏样式	`pages.json` → `globalStyle`
平台专属配置	`manifest.json`

D. 参考文档

文档	路径
产品需求文档	`docs/PRD.md`
系统架构文档	`docs/ARCH.md`
API 契约文档	`docs/backend/API_CONTRACT_v1.7.md`
页面规格说明	`docs/frontend/PAGE_SPEC_v1.8.md`
页面线框图	`docs/frontend/PAGE_WIREFRAMES_v1.8.md`
错误码注册表	`docs/backend/ERROR_CODE_REGISTRY.md`
开发启动手册	`docs/guide/DEV_KICKSTART.md`
开发与 UI 同步指南	`docs/guide/DEV_UI_SYNC_GUIDE.md`

文档版本: v1.0 | 最后更新: 2026-04-15 | 维护者: PropOS 前端团队